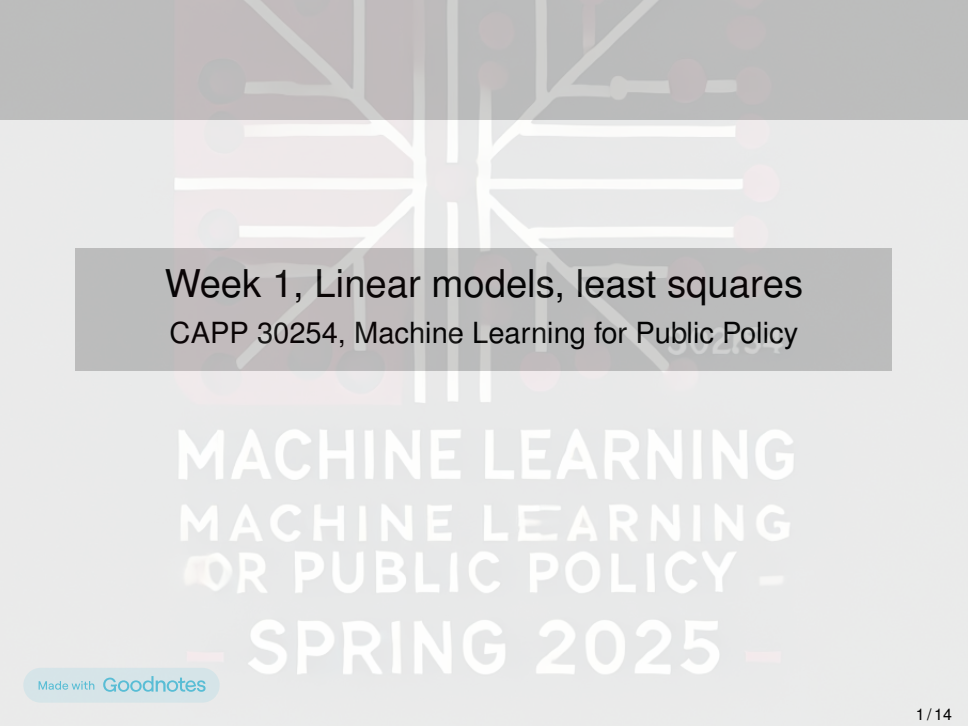
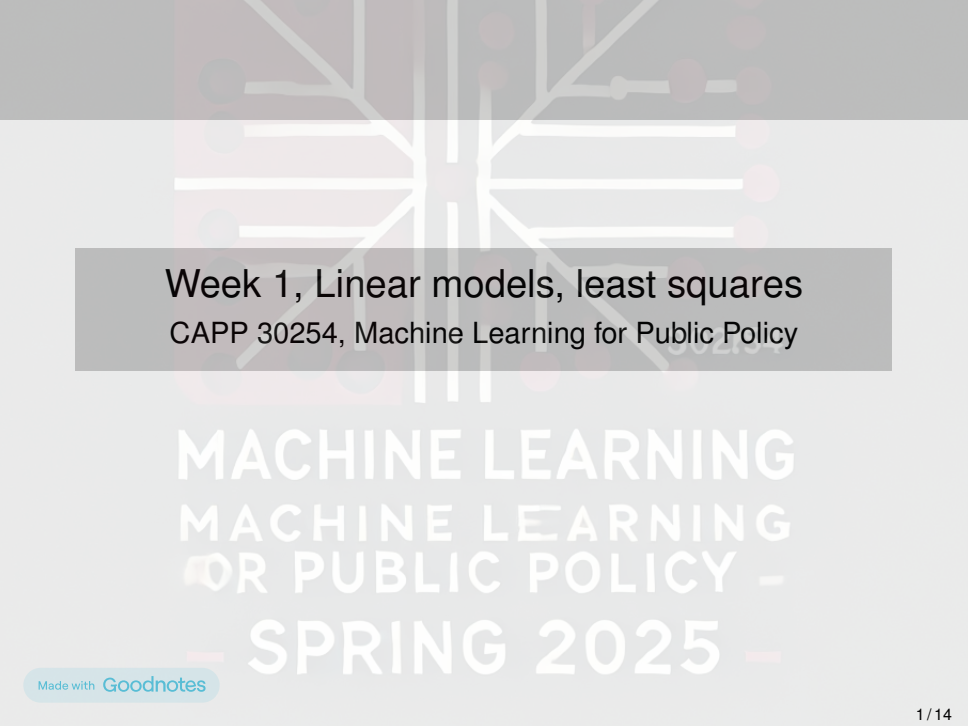




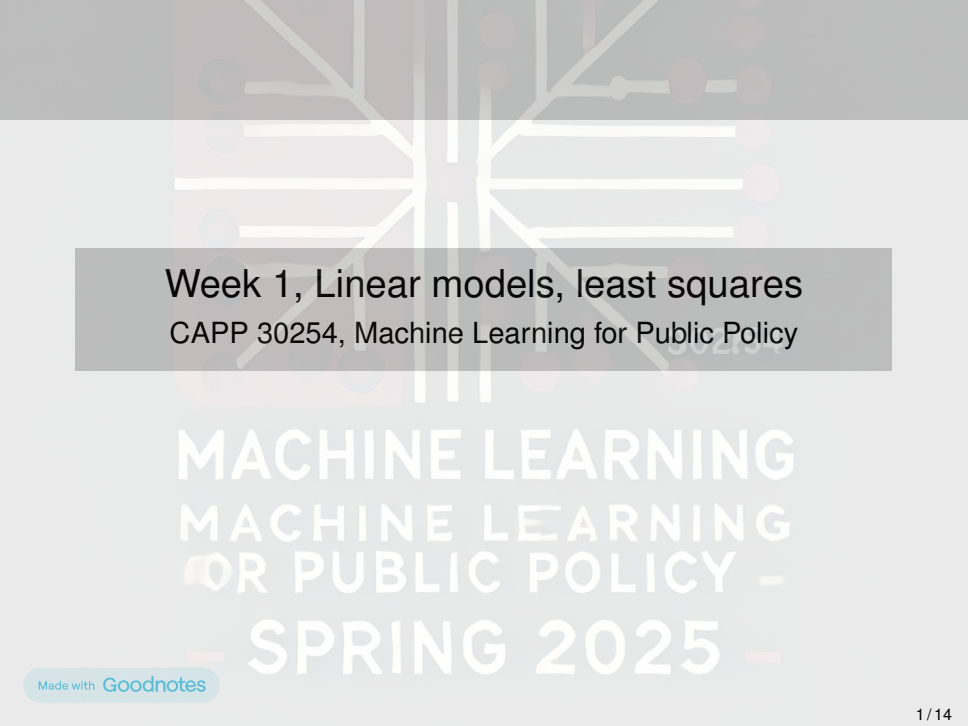
Week 1, Linear models, least squares
CAPP 30254, Machine Learning for Public Policy

MACHINE LEARNING MACHINE LEARNING FOR PUBLIC POLICY - - SPRING 2025 -



Week 1, Linear models, least squares
CAPP 30254, Machine Learning for Public Policy

MACHINE LEARNING MACHINE LEARNING FOR PUBLIC POLICY - - SPRING 2025 -



Week 1, Linear models, least squares
CAPP 30254, Machine Learning for Public Policy

MACHINE LEARNING MACHINE LEARNING FOR PUBLIC POLICY - - SPRING 2025 -

Terminology

Recall the problem where we want to train a machine learning model to determine whether a photograph is of a cat or a dog.

This is called a binary *classification* problem.

2 classes (cat|dog)



Dog



Cat



Dog



Cat



Dog

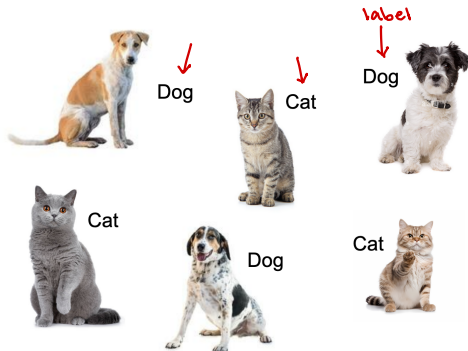


Cat

Terminology

Each example we use to train the machine learning model is *labeled* with its class.

So this is called a *supervised* binary classification problem.



Terminology

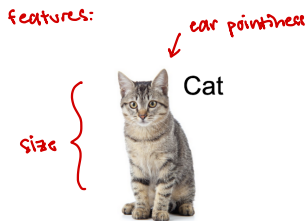
Each example we use to train the machine learning model is called a *training sample*.



Terminology

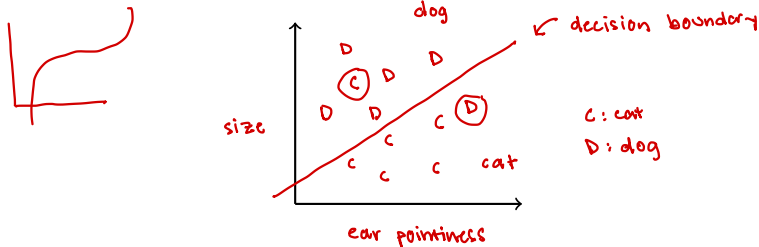
Each example we use to train the machine learning model is called a *training sample*.

The characteristics of each training sample we use to train the machine learning model are called *features*.



What does a linear machine learning model learn?

Consider the following plot of features from each training sample:

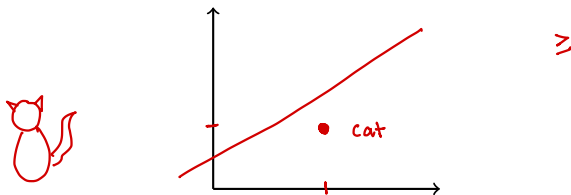


A linear machine learning model learns a line that separates data into different classes.

This line is called the *decision boundary*.

How is a linear machine learning model used?

Consider the following plot of features from each training sample:



Given new data, a linear model makes a class prediction by determining which side of the decision boundary that data falls on.

Notation

We'll use the following mathematical notation to represent a learning problem with n training samples and p features. :

■ training sample: $\mathbf{x}_i \in \mathbb{R}^p$

vectors: $\underline{y}$

■ label: $y_i \in \{+1, -1\}$

■ prediction: $\hat{y}_i$

$\underline{x}_i$ is the i th training sample $\underline{x}_i \in \mathbb{R}^p$ $\underline{x}_i = \begin{bmatrix} x_{i1} \\ \vdots \\ x_{ip} \end{bmatrix}$ ← The value of the first feature

y_i is the label of the i th training sample

$y_i = 1$ or -1 1: cat
 -1: dog

$\hat{y}_i$ is the prediction of the i th sample

$\hat{y}_i = 1$ or -1 hopefully: $\hat{y}_i = y_i$

Notation

We'll use the following mathematical notation to represent a learning problem with n training samples and p features. :

- training sample: $\mathbf{x}_i \in \mathbb{R}^p$
- label: $y_i \in \{+1, -1\}$
- prediction: $\hat{y}_i$

And to simplify things further:

- feature matrix: X
- label vector: $\mathbf{y}$

$$X = \begin{bmatrix} - & \underline{x_1^T} & - \\ & \vdots & \\ - & \underline{x_n^T} & - \end{bmatrix} \in \mathbb{R}^{n \times p}$$

row: one training sample

column: one feature

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Machine learning goal

Now we can state our goal more clearly.

Given the training samples $(\mathbf{x}_i, y_i)$ for $i = 1, \dots, n$ where $\mathbf{x}_i \in \mathbb{R}^p$ and $y_i \in \{+1, -1\}$, we want to learn a model that takes new data $\mathbf{x}$ and makes a class prediction $\hat{y}$.

^
correct

Linear machine learning model

A *linear* machine learning model makes predictions:

$$\hat{y} = \mathbf{w}^T \mathbf{x} \in \mathbb{R}$$

$$\begin{aligned} \mathbf{w}^T \mathbf{x} > 0 & \quad \hat{y} = 1 \\ \mathbf{w}^T \mathbf{x} < 0 & \quad \hat{y} = -1 \end{aligned}$$

for a weight vector $\mathbf{w}$.

The goal of a linear model is to find a good a weight vector $\mathbf{w}$.
~~~~~  
???

Prediction  $\hat{y}$  is a LG of the features of  $\mathbf{x}$ .

$$\hat{y} = w_1 x_1 + w_2 x_2 + \dots + w_p x_p$$

# Linear machine learning model

A *linear* machine learning model makes predictions:

$$\hat{y} = \mathbf{w}^T \mathbf{x} = \underline{x^T w}$$
$$= \underline{x} \cdot \underline{w}$$

for a weight vector  $\mathbf{w}$ .

The goal of a linear model is to find a weight vector  $\mathbf{w}$  so that  $\underline{Xw}$  is close to  $\mathbf{y}$ .

$$\underline{Xw} = \begin{bmatrix} -x_1^T- \\ \vdots \\ -x_n^T- \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_p \end{bmatrix} = \begin{bmatrix} \underline{x_1^T w} \\ \vdots \\ \underline{x_n^T w} \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_n \end{bmatrix} \quad \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$\underline{x_1^T w}?$   
 $\hat{y}_1$

Predictions  
with  $\underline{w}$

True label

# Least squares

A *loss function* is a function that measures how close two vectors are.

The least squares loss function  $\ell$  is:

$$\ell(\mathbf{w}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\ell$  small:  $y$  and  $\hat{y}$   
are close  
want to minimize  $\ell$

The least squares model finds a weight vector  $\mathbf{w}$  so that  $\ell$  is minimized.

Different machine learning models choose different loss functions, leading to different results.

# Least squares

The least squares weight vector must satisfy the equation:

$$X^T \mathbf{y} = X^T X \mathbf{w}$$

If the columns of  $X$  are linearly independent, we can write a closed-form expression for the least squares weight vector:

$$\ast \mathbf{w} = (X^T X)^{-1} X^T \mathbf{y}$$





1

<sup>1</sup>Generated by GPT-4, DALL-E 3 after the prompt: “Hi, could you please make a minimalistic logo for a machine learning class...” then “Could you make it less symmetrical?”